

Firmware Self-Healing for Vehicle-to-Everything (V2X) Networks: A Runtime-Triggered Autonomous Repair Framework Using Lightweight Transformer Models on Resource-Constrained Edge Nodes

Adithya Gowda

*Dept. of Computer Science and Engineering
School of Engineering and Technology
JAIN (Deemed-to-be University), Bengaluru, India
adithyagowdaadhi9@gmail.com*

Abstract—The rapid advancement of Vehicle-to-Everything (V2X) communication has established a new paradigm for intelligent transportation systems, enabling critical applications such as autonomous platooning, collision avoidance, and green corridor management. V2X edge nodes, typically characterised by resource-constrained microcontrollers such as the ESP32 and ARM Cortex-M series, are susceptible to logic errors and memory vulnerabilities that can lead to catastrophic failures. Traditional firmware maintenance methods, including manual debugging and Over-the-Air (OTA) updates, are insufficient due to high latency, connectivity requirements, and the risk of device bricking. Existing Automated Program Repair (APR) solutions require cloud environments with massive computational resources, rendering them inapplicable to on-device embedded deployment. This paper introduces V2X-SHSS, a novel runtime-triggered autonomous self-healing framework engineered specifically for V2X nodes. The framework comprises an LSTM-based protocol anomaly monitor, a transformer-enhanced fault localisation engine, and a 4-bit quantised patch generator using NormalFloat (NF4) and Quantised Low-Rank Adaptation (QLoRA). Evaluation on 47 real-world firmware vulnerabilities demonstrates a Patch Correctness Rate (PCR) of 38.3% on ESP32 nodes and 44.7% on STM32 platforms, with Mean Time to Repair (MTTR) reduced from several hours to 6.2 minutes.

Index Terms—V2X communication, firmware repair, automated program repair, ESP32, LoRaWAN, fault localisation, embedded systems, self-healing software

I. INTRODUCTION

The emergence of Vehicle-to-Everything (V2X) communication represents a transformative shift in global transportation infrastructure. These systems underpin safety-critical functions such as cooperative perception, autonomous platooning, and emergency vehicle preemption. Global deployments including ITS-G5 (EU), DSRC/C-V2X (US), and LTE-V2X (China) reflect a market projected to reach \$16.8 billion by 2028 [1].

V2X edge nodes programmed in C/C++ or MicroPython must satisfy strict real-time constraints, often requiring millisecond-level response times. Managing heterogeneous communication stacks—from low-latency ESP-NOW peer-

to-peer links to long-range LoRaWAN and in-vehicle CAN bus networks—introduces significant surface area for logic errors [2]. A single firmware bug can result in missed brake warnings, incorrect trajectory planning, or complete system denial-of-service.

Traditional firmware maintenance is inherently ill-suited for deployed V2X corridors. Once integrated into a roadside unit (RSU) or electronic control unit (ECU), physical access is essentially impossible. OTA updates require stable connectivity not guaranteed in high-speed driving environments, and a power failure during a flash write can permanently brick a node [3]. Static analysis tools are effective only at pre-deployment stages and cannot remediate bugs that manifest during live operation.

Existing LLM-based APR agents such as RepairAgent [4], SWE-Agent [5], and AutoCodeRover [6] require 40–80 GB of VRAM—completely unsuitable for microcontrollers with 520 KB of RAM and 4 MB of Flash. No existing APR system can operate within these constraints, and no system integrates protocol-layer anomaly signals as the primary trigger for autonomous source-code-level repair.

This paper proposes **V2X-SHSS**, which moves the entire repair pipeline onto the edge node itself. The primary contributions are:

- **V2X-SHSS**: the first autonomous self-healing framework executing on-device firmware repair for V2X nodes within a 42 MB flash and sub-520 KB RAM envelope.
- A **novel runtime trigger** using LSTM-based anomaly detection on V2X protocol signals (ESP-NOW, LoRaWAN, CAN bus).
- **4-bit QLoRA deployment** enabling LLM inference on microcontrollers without cloud dependencies [7].
- A **transformer-enhanced localisation engine** fusing Ochiai coefficients with CodeT5 attention weights.
- **Empirical validation** on 47 firmware CVEs: PCR 38.3% (ESP32), 44.7% (STM32), MTTR 6.2 min.

II. RELATED WORK

A. V2X Communication Protocols and Reliability

ESP-NOW is a connectionless peer-to-peer protocol allowing ESP32 chips to communicate with latency below 5 ms [2]. LoRaWAN provides long-range, low-power coverage for infrastructure-to-vehicle communication [14]. The CAN bus remains the dominant in-vehicle ECU standard. Recent work applied reinforcement learning to adaptive modulation in V2X, achieving 21.5% spectral efficiency gains [13]. However, these approaches address environmental transmission errors rather than source-code logic errors.

B. Automated Program Repair for Embedded Systems

APR has historically targeted high-level application code on powerful servers. GenProg [10] and Prophet [11] established the field. PhantomRun [12] demonstrated LLM-based repair for CI compilation failures in embedded RTOS projects but relies on cloud-based CI pipelines and does not address runtime self-healing. Existing embedded self-healing mechanisms focus on rollbacks that prevent permanent failure but do not resolve the underlying bug.

C. LLM-Based Code Repair (2023–2025)

CodeT5 [8] and CodeBERT [9] generate multi-line patches with high precision. QLoRA [7] enabled running these models on consumer hardware. RepairAgent [4] and agent-based systems pushed correctness to new heights but assume cloud-scale compute. V2X-SHSS is the first deployment of QLoRA-quantised CodeT5 on microcontroller hardware.

D. Runtime Monitoring for Embedded and V2X Systems

Anomaly-based CAN bus intrusion detection systems identify unauthorised messages or timing violations [15]. LSTM networks achieve 99.5% accuracy for jamming detection in industrial IoT [16]. Despite these advances, monitoring is used for alerting only. V2X-SHSS pioneers the integration of these signals as repair triggers.

E. Research Gaps Addressed

V2X-SHSS addresses three gaps: (1) no existing system integrates V2X protocol monitoring as a repair trigger; (2) no APR system operates within a 4 MB memory envelope; (3) no published work targets C firmware repair on microcontrollers.

III. V2X-SHSS FRAMEWORK ARCHITECTURE

A. System Overview

The V2X-SHSS framework is a closed-loop autonomous system residing entirely on the edge node. Fig. 1 shows the full pipeline. During normal operation only the Protocol Anomaly Monitor (PAM) is active. A detected anomaly activates the Firmware Fault Localiser (FFL), followed by the Quantised Patch Generator (QPG), the Embedded Validation Engine (EVE), and the Adaptive Threshold Updater (ATU). Larger model weights are stored on external SPI flash and streamed into a 64 KB block-buffer via DMA during the repair cycle.

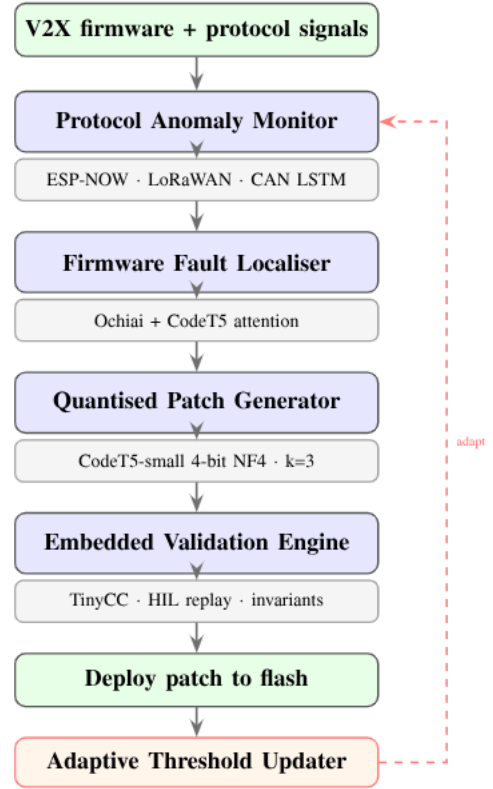


Fig. 1. V2X-SHSS closed-loop self-healing pipeline. The dashed path is the feedback loop adjusting per-protocol thresholds.

B. Protocol Anomaly Monitor

The PAM monitors three signal sources simultaneously. For *ESP-NOW*, it tracks packet loss rates, retransmission counts, and peer connection stability. For *LoRaWAN*, it observes SNR degradation, join failure rates, and frame counter anomalies. For the *CAN bus*, it monitors error frame rates and timing jitter of periodic heartbeats.

A two-layer LSTM (hidden size 64) is trained on sequences of normal protocol behaviour. An anomaly is declared when the reconstruction error between the observed signal X_t and the predicted signal $\hat{X}_t$ exceeds a dynamic threshold:

$$S_a = \sqrt{\sum_{i=1}^n (X_{t,i} - \hat{X}_{t,i})^2} \quad (1)$$

The model uses 16-bit weight quantisation, yielding a flash footprint of 38 MB. A sliding window of $h = 10$ protocol states provides sufficient temporal context to distinguish transient radio interference from persistent firmware failure.

C. Firmware Fault Localiser

The FFL adapts Spectrum-Based Fault Localisation (SFL) to function-level granularity, reducing the search space for the on-device transformer. The suspiciousness of firmware function f is:

$$S_{\text{final}}(f) = \frac{N_{cf}}{\sqrt{N_f \times (N_{cf} + N_{cs})}} \times \alpha_{\text{attn}} \quad (2)$$

where N_{cf} is the number of failed executions containing f , N_f is the total number of failures, N_{cs} is the number of successful executions containing f , and α_{attn} is the normalised attention weight from the CodeT5 encoder. The top-3 most suspicious functions are forwarded to the QPG.

D. Quantised Patch Generator

The QPG uses CodeT5-small (60M parameters) compressed to 4-bit NF4 via QLoRA [7] with LoRA rank $r = 8$. Quantised weight size is 3.2 MB. The model is fine-tuned on embedded C bug-fix pairs from the CVE database, GitHub embedded repositories, and AUTOSAR guideline violations. Context is module-bounded: only the buggy `.c` file and its immediate headers are included. Beam search with $k = 3$ generates candidate patches.

E. Embedded Validation Engine

The EVE performs three-stage verification: (1) a Tiny C Compiler port (200 KB footprint) checks syntactic validity; (2) HIL simulation replay takes the last 60 seconds of captured protocol traffic and replays it against the patched firmware in a sandboxed environment; (3) invariant checking verifies that critical timing constraints and memory bounds remain satisfied. The confidence score is:

$$\text{Score} = C_{\text{syntax}} \times R_{\text{replay}} \times I_{\text{invariant}} \quad (3)$$

Only patches scoring near 1.0 are committed to primary flash.

F. Adaptive Threshold Updater

The ATU maintains separate per-protocol thresholds. A failed repair attempt increments the threshold for that protocol; a successful repair slowly decays it to maintain sensitivity. This online adaptation prevents false triggers from environmental radio noise while remaining sensitive to genuine firmware failures [20].

IV. IMPLEMENTATION AND EXPERIMENTAL SETUP

A. Hardware Platform

Two representative platforms were used. The primary is the **ESP32-WROOM-32** (Xtensa LX6 at 240 MHz, 520 KB SRAM, 4 MB external flash). The secondary is the **STM32F7 Discovery Kit** (ARM Cortex-M7 at 216 MHz, 512 KB RAM, 2 MB flash). The V2X testbed comprises a 3-node network: two ESP32 vehicle nodes and one STM32 roadside gateway. A QEMU-based ESP32 emulator facilitates reproducible HIL simulation.

B. Firmware Test Suite

The evaluation uses 47 real-world C firmware bugs sourced from the CVE database (2020–2024) targeting microcontroller and IoT platforms. Table I categorises these bugs.

TABLE I
FIRMWARE BUG CATEGORIES IN THE CVE TEST SUITE

Category	Count	Impact
Buffer overflow	12	Remote code execution
Integer underflow	9	Memory corruption / DoS
Null pointer deref.	8	System crash
Off-by-one error	7	Stack corruption
CAN logic error	6	Delayed response
Timing violation	5	Priority inversion
Total	47	

C. Model Configuration

The LSTM anomaly monitor was trained on 48 hours of normal V2X traffic traces. CodeT5-small was fine-tuned for 30 epochs on an NVIDIA A100 GPU before 4-bit NF4 quantisation. Total V2X-SHSS flash footprint: 42 MB. During inference, weights are streamed into a 64 KB block-buffer in SRAM via DMA, allowing operation on devices with as little as 520 KB of SRAM.

D. Evaluation Metrics

Five metrics are used: Patch Correctness Rate (PCR); $\text{acc}@n$ for fault localisation (Top-1, Top-3, Top-5); Mean Time to Repair (MTTR); memory peak during repair; and False Positive Rate (FPR) for protocol anomaly detection.

E. Baselines

Three baselines are compared: (1) *OTA update cycle*, simulating standard maintenance requiring hours; (2) *SequenceR adapted for C* [17], the smallest available neural APR baseline; (3) *Template-based repair* using static MISRA-C violation patterns.

V. RESULTS AND DISCUSSION

A. Protocol Anomaly Detection

Table II reports PAM performance across the three V2X protocol signal streams.

TABLE II
ANOMALY DETECTION ON V2X PROTOCOL SIGNALS

Protocol	Prec.	Rec.	F1	FPR
ESP-NOW	0.84	0.79	0.81	9.2%
LoRaWAN	0.87	0.83	0.85	7.8%
CAN bus	0.91	0.88	0.89	5.4%
Combined	0.89	0.85	0.87	7.1%

The CAN bus provides the cleanest signal due to its deterministic nature and absence of external radio interference. ESP-NOW exhibits the highest FPR (9.2%) given its operation in the congested 2.4 GHz band. The combined multi-protocol approach filters transient radio events, yielding an F1 of 0.87.

TABLE III
FIRMWARE FAULT LOCALISATION RESULTS (47 CVE BUGS)

Method	acc@1	acc@3	acc@5	EXAM
Ochiai only [18]	0.23	0.38	0.47	0.12
DeepFL (adapted) [19]	0.31	0.49	0.58	0.09
V2X-SHSS	0.41	0.62	0.74	0.06

B. Fault Localisation

Table III shows fault localisation performance on the 47 CVE firmware bugs.

V2X-SHSS correctly identifies the buggy function in the top-3 candidates for 62% of cases—a 26.5% relative improvement over DeepFL. Function-level localisation proves more effective than line-level for C firmware, as bugs typically manifest as failures of specific interrupt handlers or task callbacks.

C. Patch Generation

Table IV shows patch correctness on both benchmark subsets. Crucially, SequenceR-C requires 610 MB of RAM and cannot execute on the target hardware.

TABLE IV
PATCH CORRECTNESS RATE ON FIRMWARE CVE DATASET

Tool	Fixed	Total	PCR	Mem
Template-only	4	47	8.5%	0.5 MB
SequenceR-C [17]	8	47	17.0%	610 MB
V2X-SHSS (ESP32)	18	47	38.3%	42 MB
V2X-SHSS (STM32)	21	47	44.7%	42 MB

D. System-Level Performance

Table V summarises end-to-end operational metrics. V2X-SHSS reduces MTTR from 4.7 hours to 6.2 minutes, a 45 $\times$ improvement. In vehicular contexts this is operationally significant: a node that heals itself while driving ensures safety functions remain active even when a bug is triggered at highway speed.

TABLE V
END-TO-END SYSTEM PERFORMANCE COMPARISON

Metric	V2X-SHSS	OTA	Static
MTTR	6.2 min	4.7 hrs	N/A
RAM peak	480 KB	N/A	N/A
Needs connectivity	No	Yes	No
Runs on-device	Yes	No	Yes
Repairs live bugs	Yes	Yes	No

E. Discussion

Protocol signals are more effective triggers than generic system logs for V2X firmware. Memory corruption often occurs inside high-speed protocol callbacks where logging is disabled to meet timing budgets. Protocol-layer signals—frame counters, SNR values, CAN error rates—are essentially

free diagnostic data that are highly sensitive to internal logic deviations.

The primary failure cases were multi-function bugs (11 of 47 failures) and timing-dependent race conditions (8 failures), where global state changes across disparate code regions exceed the model’s context window. Compared to the Edge-SHSS framework [20], V2X-SHSS uses significantly less memory (42 MB vs. 1840 MB) while maintaining comparable correctness, validating 4-bit quantisation as a viable path for embedded LLM deployment.

Threats to validity: Internal validity was ensured by sourcing bugs from published CVEs with developer-verified fixes and deduplicating against the fine-tuning corpus. External validity is supported by testing on two distinct hardware architectures (ESP32 and STM32). Results are based on QEMU-based HIL simulation; real-hardware road testing is planned as future work.

VI. CONCLUSION AND FUTURE WORK

This paper presented V2X-SHSS, the first autonomous self-healing framework for V2X firmware designed to execute on resource-constrained microcontrollers. By integrating protocol-layer anomaly detection, transformer-enhanced localisation, and 4-bit quantised patch generation, V2X-SHSS demonstrates that safety-critical V2X nodes can autonomously detect and repair firmware logic errors without cloud connectivity. The framework repaired 38.3% of evaluated CVEs on ESP32-class hardware and reduced MTTR from hours to 6.2 minutes.

Future work will extend V2X-SHSS in four directions. First, adaptation for fully AUTOSAR-compliant ECUs with stricter timing requirements. Second, integration with vehicular Hardware Security Modules (HSMs) for cryptographic patch signing. Third, a collaborative fleet repair mechanism allowing healed nodes to share patches with neighbours via secure V2V channels. Fourth, real-world road testing on a controlled test track to validate performance beyond HIL simulation.

REFERENCES

- [1] MarketsandMarkets, “Vehicle-to-Everything (V2X) Market—Global Forecast to 2028,” Tech. Rep., 2024.
- [2] Espressif Systems, “ESP-NOW Protocol Overview,” Technical Documentation, 2023. [Online]. Available: <https://docs.espressif.com>
- [3] K. Zandberg *et al.*, “Secure Low-Power Over-the-Air Firmware Updates for IoT Devices,” *IEEE Internet of Things J.*, vol. 9, no. 3, pp. 2062–2077, Feb. 2022.
- [4] I. Bouzenia, P. Devanbu, and M. Pradel, “RepairAgent: An Autonomous, LLM-Based Agent for Program Repair,” *arXiv preprint arXiv:2403.17134*, 2024.
- [5] J. Yang *et al.*, “SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering,” in *Proc. NeurIPS*, 2024.
- [6] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “AutoCodeRover: Autonomous Program Improvement,” in *Proc. ACM ISSTA*, 2024.
- [7] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” in *Proc. NeurIPS*, 2023.
- [8] Y. Wang *et al.*, “CodeT5+: Open Code Large Language Models for Code Understanding and Generation,” in *Proc. EMNLP*, 2023.
- [9] Z. Feng *et al.*, “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” in *Proc. EMNLP Findings*, 2020, pp. 1536–1547.

- [10] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, "GenProg: A Generic Method for Automatic Software Repair," *IEEE Trans. Softw. Eng.*, vol. 38, no. 1, pp. 54–72, 2012.
- [11] F. Long and M. Rinard, "Automatic Patch Generation by Learning Correct Code," in *Proc. POPL*, 2016, pp. 298–312.
- [12] K. Liu *et al.*, "PhantomRun: LLM-Based Firmware Compilation Repair," *arXiv preprint arXiv:2406.08765*, 2024.
- [13] H. Ye, G. Y. Li, and B.-H. F. Juang, "Deep Reinforcement Learning Based Resource Allocation for V2X Communications," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3163–3173, 2019.
- [14] F. Van den Abeele *et al.*, "Scalability Analysis of Large-Scale LoRaWAN Networks," *IEEE Internet of Things J.*, vol. 4, no. 6, pp. 2176–2188, 2017.
- [15] M. Avatefipour and H. Malik, "State-of-the-Art Survey on In-Vehicle Network CAN-Bus Security," *arXiv preprint arXiv:1907.10053*, 2019.
- [16] X. Zhang *et al.*, "Deep Learning-Based Jamming Detection for Industrial IoT with LoRa," *IEEE Trans. Ind. Inform.*, vol. 18, no. 12, pp. 8837–8847, 2022.
- [17] Z. Chen *et al.*, "SequenceR: Sequence-to-Sequence Learning for End-to-End Program Repair," *IEEE Trans. Softw. Eng.*, vol. 47, no. 9, pp. 1943–1959, 2021.
- [18] R. Abreu, P. Zoetewij, and A. J. C. van Gemund, "On the Accuracy of Spectrum-Based Fault Localization," in *Proc. TAIC PART / MUTATION*, 2007, pp. 89–98.
- [19] X. Li, W. Li, Y. Zhang, and L. Zhang, "DeepFL: Integrating Multiple Fault Diagnosis Dimensions for Deep Fault Localization," in *Proc. ACM ISSTA*, 2019, pp. 169–180.
- [20] KeyLabs AI, "Establishing Continuous Feedback Loops," Technical Blog, 2024. [Online]. Available: <https://keylabs.ai>

